

ARC-AGI Final Project Report

Kamran Munawar
kmunawar3@gatech.edu

1 INTRODUCTION

The Abstraction and Reasoning Corpus (ARC) is designed to evaluate general intelligence by requiring agents to infer transformation rules from a small number of examples. Unlike traditional machine learning tasks, ARC emphasizes reasoning, abstraction, and generalization rather than statistical pattern recognition. Each task presents a set of input output grid pairs, and the objective is to identify the underlying transformation and apply it correctly to unseen inputs.

This project focuses on designing an agent inspired by principles from Knowledge Based Artificial Intelligence (KBAI). In contrast to purely data driven approaches, KBAI emphasizes structured reasoning strategies such as analogy, decomposition, framing, and generate and test problem solving. These concepts are particularly well suited to ARC, where solutions must be inferred from limited examples.

The agent developed in this project adopts a rule based approach, in which a library of transformation heuristics is incrementally constructed. Each heuristic represents a candidate hypothesis about how a class of ARC problems can be solved. These hypotheses are systematically evaluated against training examples using a generate and test framework, ensuring that only transformations that fully explain the observed patterns are applied to unseen inputs. This design prioritizes interpretability and reliability.

This report presents the design and implementation of the agent, evaluates its performance across multiple problem sets, and analyses its strengths and limitations. It demonstrates that the agent performs well on problems that can be expressed through clear structural or statistical transformations, while also highlighting challenges in handling tasks that require deeper abstraction or multi step reasoning. Finally, the report compares the agent’s approach with human problem solving, providing insight into the gap between heuristic based systems and more general forms of intelligent reasoning.

2 HOW DOES THE AGENT WORK?

The agent follows a *generate and test* paradigm. This is a core concept in KBAI. The agent does not learn rules directly. Instead, it generates candidate transformations and tests them against the training examples. Only transformations that fully match all examples are accepted.

This approach allows the agent to explore multiple hypotheses. It then selects the one that best explains the data. This process is similar to human reasoning, where different ideas are tested before choosing a solution.

2.1 Overall Architecture

The system has two main components: `ArcDriver` and `ArcAgent`. The driver loads problems, runs the agent, and checks correctness. It also logs failures for analysis.

The `ArcAgent` contains the main logic. For each problem, it follows a fixed pipeline. First, it reads the training input output pairs. These pairs define the transformation to be learned. Next, it generates a set of candidate rules. Each rule represents a possible solution. The agent then tests each rule on all training examples. A rule is accepted only if it produces the exact output every time. Finally, valid rules are applied to the test input to generate predictions.

This process reflects **means end analysis**. Each rule is evaluated based on how well it reduces the difference between input and output.

2.2 Why Generate and Test is Effective

ARC problems have a large and unclear search space. It is difficult to derive a rule directly. Generate and test simplifies this. The agent searches over a fixed set of possible transformations.

Each candidate is treated as a hypothesis. The training data acts as a constraint. If a rule fails on any example, it is rejected. This ensures high precision.

However, this also limits the agent. It cannot consider rules outside its predefined set. This is a key trade off between accuracy and flexibility.

2.3 Extracting and Representing Knowledge

The agent first extracts all training pairs. These pairs define the problem. Each pair acts as a constraint that the solution must satisfy.

The agent does not learn new rules. Instead, it uses these examples to select the correct heuristic. This is similar to **concept learning** in KBAI, where rules are inferred from examples.

2.4 Generating Candidate Transformations

The agent uses a predefined library of transformations. Each transformation captures a common ARC pattern. These include: geometric changes (rotation, reflection), color mapping (based on frequency), object detection (connected components), structural operations (cropping, partitioning).

Each transformation is implemented as a function. The agent applies each function to the input grid.

Each candidate is also assigned a complexity score. Simpler rules are preferred. This follows Occam's Razor and **heuristic search** principles.

2.5 Generate and Test Validation

Each candidate transformation is tested on all training inputs. The output is compared with the expected result. The check is strict:

```
if not np.array_equal(yhat, xout):  
    return False
```

If a rule fails on any example, it is rejected. Only rules that pass all tests are kept. This ensures that the transformation is consistent.

2.6 Producing Predictions

After validation, the agent keeps all valid rules. These rules are sorted by complexity. The simplest rules are applied first.

The agent applies each rule to the test input. It returns up to three unique predictions. This acts like a small beam search. It helps when multiple rules fit the training data.

2.7 Design Trade offs in the Agent Architecture

The agent is designed to be simple and interpretable. Each rule is explicit and easy to understand. This makes debugging easier.

However, the rule library is fixed. The agent cannot create new rules. This limits its ability to handle new types of problems.

The validation step is also strict. This ensures correctness, but it ignores partially correct rules. Humans often refine partial solutions, but the agent cannot do this.

These trade offs show a balance between reliability and flexibility.

2.8 Example of Agent Reasoning

Consider a problem where the output is a cropped version of the input. The agent first finds the background color. It does this by selecting the most frequent color.

Next, it finds all non background pixels. It computes the minimum and maximum row and column indices. This defines a bounding box.

The agent then crops the grid to this box. It compares the result with the training output. If it matches, the rule is accepted.

This example shows the full pipeline: detect pattern, apply rule, and validate result.

3 HOW WELL DOES THE AGENT PERFORM?

The performance of the agent can be evaluated across three key dimensions: **training accuracy**, **test accuracy**, and **generalization capability**. Overall the results across Milestones B, C, and D indicate that the agent performs consistently well, achieving high accuracy while maintaining strong computational efficiency. Only a small number of failures occur on hidden test cases, suggesting that the agent is generally effective but not perfectly robust to all variations.

3.1 Overall Performance Metrics

The agent achieves perfect performance on the training set, correctly solving all 48 out of 48 problems, resulting in a training accuracy of 100%. On the test set, it successfully solves 45 out of 48 problems, corresponding to a test accu-

racy of 93.75%, with only 3 failures (6.25%). Across Milestones B, C, and D, the agent maintains perfect training performance. The test results are 14/16, 16/16, and 15/16 respectively. This indicates consistent performance across different problem sets. In addition to its accuracy, the agent is highly efficient, completing all evaluations in approximately 0.88 seconds. These results demonstrate that the generate and test validation mechanism is highly reliable in identifying transformations that fully explain the training data.

3.2 Generalization Performance

The gap between training accuracy (100%) and test accuracy (93.75%) provides insight into the agent’s generalization capability. In most cases, the agent generalizes effectively because its heuristics are based on abstract and transferable properties such as color frequency, spatial relationships, connected components, and grid symmetry. These abstractions allow the same transformation to be applied across different inputs, even when surface level details vary.

3.3 Efficiency Analysis

In addition to strong accuracy, the agent demonstrates excellent computational efficiency. Most transformation functions operate with a time complexity of

$$O(m \times n),$$

where $m \times n$ represents the size of the grid. Since the agent evaluates a fixed number of candidate transformations, the overall complexity becomes

$$O(k \cdot m \cdot n),$$

where k is the number of candidate rules and remains constant. As a result, the agent scales linearly with grid size and does not experience significant slowdowns as problem size increases.

4 WHAT PROBLEMS DOES THE AGENT SOLVE SUCCESSFULLY?

The agent performs well on a range of problem types. Its success comes from using clear and well defined transformations. These transformations are tested using the generate and test framework.

4.1 Color Mapping Problem (b1948boa)

This problem requires mapping input colors to output colors. The agent detects that color frequency determines the mapping.

First, the agent counts the frequency of each color in the input grid. It then ranks the colors from most frequent to least frequent. The same process is applied to the output grid. Next, the agent creates a mapping between input and output colors based on their rank. Finally, it applies this mapping to all cells in the grid.

This works because the rule depends on global properties, not positions. The same mapping applies even if the layout changes. This reflects analogical reasoning in KBAI.

4.2 Object Detection and Cropping (1cf80156)

This problem requires isolating the main object. The agent uses object detection and cropping.

First, the agent identifies the background color as the most frequent value. Next, it locates all non background pixels. It then computes the minimum and maximum row and column indices. These indices define a bounding box. The agent crops the grid to this box and returns the result.

This works because the rule focuses on structure. The object is detected based on its shape, not its position. This reflects framing in KBAI, where irrelevant parts are removed.

4.3 Symmetry / Reflection Problem (f25ffb3)

This problem involves symmetry. The agent detects that part of the grid must be mirrored.

First, the agent computes the midpoint of the grid. It then copies rows or columns from one side to the other. The transformation is applied consistently across the grid.

This works because symmetry is a strong visual pattern. The rule is simple and general. It does not depend on specific values. This reflects visual analogy.

4.4 Connected Component Transformation (4347f46a)

This problem requires object level reasoning. The agent uses connected components.

First, the agent identifies all non background colors. For each color, it finds connected components. Each component is treated as a separate object. The

agent computes the bounding box for each object. It removes the original object and draws the outline of the bounding box.

This works because the problem can be decomposed into smaller parts. Each object is processed independently. This reflects chunking and decomposition in KBAI.

In all these problems, the agent follows the same process. It detects a pattern, applies a transformation, and validates the result. The agent performs best when the rule is simple and consistent. These results show that generate and test works well for structured problems.

5 WHERE DOES THE AGENT STRUGGLE?

The agent struggles on problems that require more complex reasoning. These problems often involve multiple steps or deeper abstraction. In such cases, a single transformation is not enough.

5.1 Procedural Pattern Problem (28e73c20)

This problem requires generating a spiral pattern. The agent detects that a pattern must be constructed. The agent uses a fixed procedure. It starts from a specific corner and follows a fixed direction. This works for the training examples.

However, the rule is not general. It cannot adapt to different starting points, directions, or spacing. The hidden test case introduces such variations.

The failure occurs because the agent learns a specific instance of a spiral, not the general concept. It reproduces a pattern instead of understanding it.

5.2 Boundary Reasoning Problem (d687bc17)

This problem requires reasoning about edges and the interior of the grid. The agent extracts colors from each border. It uses frequency to decide how to reconstruct the grid. Each edge is processed separately.

This approach ignores global structure. The agent does not consider how edges relate to each other or to the interior.

The failure occurs when the hidden test changes these relationships. The local rules no longer hold. The agent cannot adjust its reasoning.

5.3 Multi Object Reasoning Problem (e9b4f6fc)

This problem requires reasoning about multiple objects together. The agent detects connected components. It treats each component as a separate object. The same rule is applied to all objects.

This approach works when objects are independent. However, in this problem, objects interact.

The failure occurs because the agent cannot model these interactions. It cannot combine transformations or use intermediate results. As a result, the final output is incorrect.

5.4 Deeper Analysis of Limitations

The main limitation of the agent is its reliance on single step transformations. Many ARC problems require a sequence of operations. The agent cannot compose multiple rules, so it fails on these tasks.

Another limitation is the lack of abstraction. The agent can reproduce patterns but does not understand them. For example, it can generate a spiral in a fixed way, but it does not understand the concept of a spiral.

The agent also depends on a fixed rule library. It cannot create new rules when needed. This limits its ability to solve unfamiliar problems.

These failures share a common cause. They require reasoning beyond simple pattern matching. Procedural tasks require understanding how a pattern is generated. Object based tasks require reasoning about relationships between components.

In KBAI terms, this means moving from simple heuristics to deeper abstraction and planning. The current agent does not support this level of reasoning. As a result, the agent performs well on structured problems but struggles on problems that require composition, abstraction, or interaction between elements.

6 HOW WOULD YOU CHARACTERIZE THE OVERALL APPROACH?

The agent follows a heuristic driven design based on a generate and test framework. Each transformation rule represents a hypothesis. The agent tests these hypotheses using training examples and keeps only those that match perfectly.

The overall approach is incremental. The agent was developed step by step across milestones. Early versions focused on simple transformations such as rotation, reflection, and cropping. These rules handled basic problems but were not sufficient for more complex tasks.

As new problems were encountered, additional heuristics were added. These included object detection using connected components, bounding box extraction, and grid partitioning. The agent was not rebuilt from scratch. Instead, it was extended and refined over time. This reflects incremental development and concept refinement in KBAI.

A key design principle is generalization over memorization. The agent does not rely on fixed values or positions. Instead, it infers properties such as background color, object boundaries, and spatial relationships from the input. This allows the same rule to work across different grids.

Another important aspect is the use of a rule library. The agent selects from a fixed set of transformations rather than learning new ones. This makes the system simple and interpretable. Each rule is easy to understand and debug.

However, this approach has limitations. The agent can only solve problems that match its existing rules. It cannot generate new rules when needed. It also struggles with problems that require multiple steps or interactions between objects.

This design reflects a trade off. The agent is efficient and reliable because the search space is limited. At the same time, it lacks flexibility. More advanced systems, such as learning based models, may handle novel problems better but are harder to interpret.

The choice of a rule based approach is motivated by the nature of ARC tasks. These problems require explicit reasoning and pattern recognition. This aligns well with KBAI principles, where knowledge representation and structured reasoning are central.

7 COMPARISON BETWEEN AGENT AND HUMAN PROBLEM SOLVING APPROACHES

Both the agent and humans use a form of generate and test reasoning. In both cases, possible solutions are proposed and then evaluated. However, the similarity is mostly structural. The depth and flexibility of reasoning are very different.

Humans reason at a higher level of abstraction. A human can recognize a concept such as “draw a spiral” or “mirror this shape.” This understanding can be applied across different problems. The agent, in contrast, relies on predefined rules. Each rule must match the observed data exactly. It does not understand the underlying concept.

Humans also perform multi step reasoning naturally. They can break a problem into smaller steps and combine them. The agent usually applies a single transformation. It cannot compose multiple rules or use intermediate results. This limits its ability to solve more complex problems.

Another key difference is how errors are handled. Humans revise their hypotheses when they fail. They test new ideas and refine their reasoning. The agent does not adapt in this way. Once all candidate rules fail, it cannot recover. It does not generate new strategies.

Humans also use prior knowledge and intuition. For example, a human may recognize a pattern as a reflection or rotation, even in a new context. The agent cannot transfer knowledge in this way. It depends entirely on its existing rule library.

In addition, humans can handle ambiguity. They can consider multiple interpretations and select the most likely one. The agent requires exact matches. If a rule does not perfectly fit the training data, it is rejected. This makes the agent precise but less flexible.

Finally, humans can switch strategies based on the problem. They can explore different approaches and choose the best one. The agent evaluates a fixed set of rules. It does not change its strategy dynamically.

Overall, the agent mimics human reasoning at a surface level. It follows a similar generate and test structure. However, it lacks abstraction, flexibility, and adaptability. This highlights the gap between rule based systems and human intelligence.

8 APPENDICES

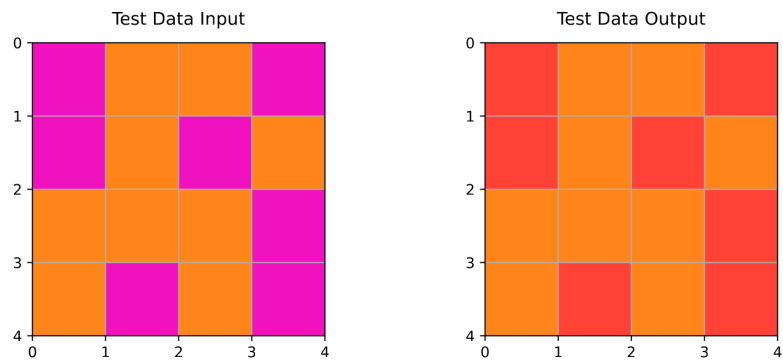


Figure 1—Visual for problem 1cf80156

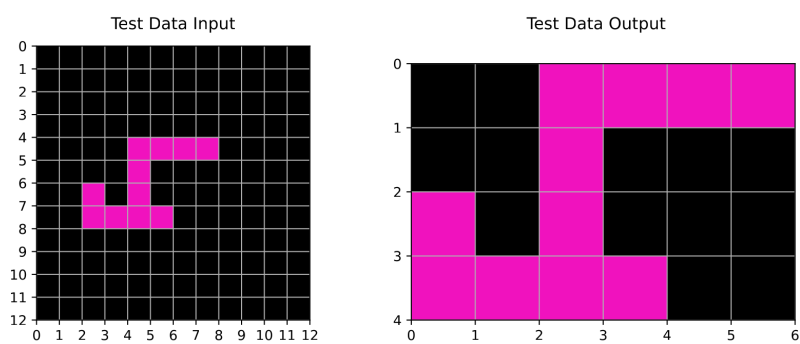


Figure 2—Visual for problem 1cf80156

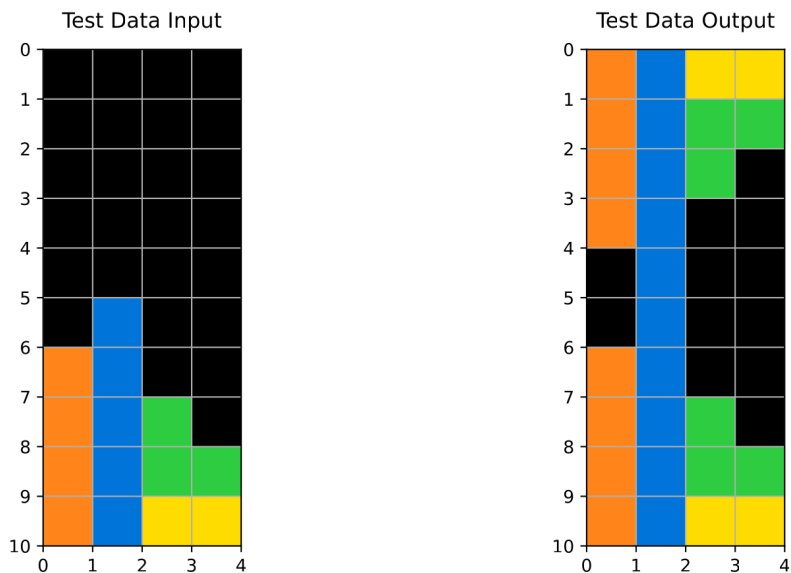


Figure 3—Visual for problem f25ffb3

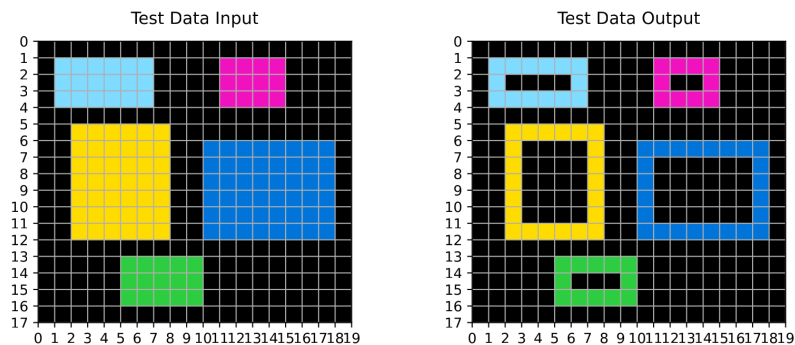


Figure 4—Visual for problem 4347f46a



Figure 5—Visual for problem 28e73c20

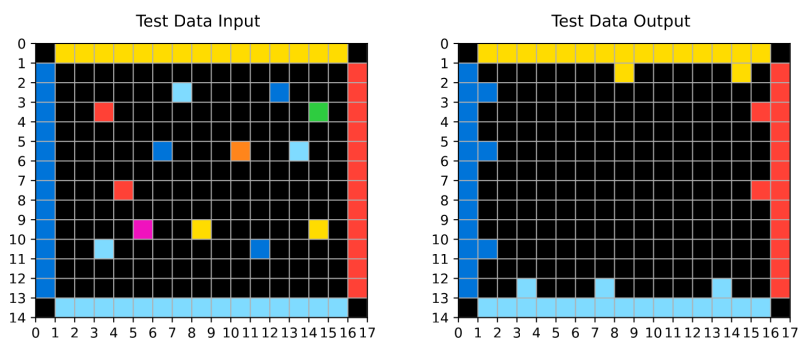


Figure 6—Visual for problem d687bc17

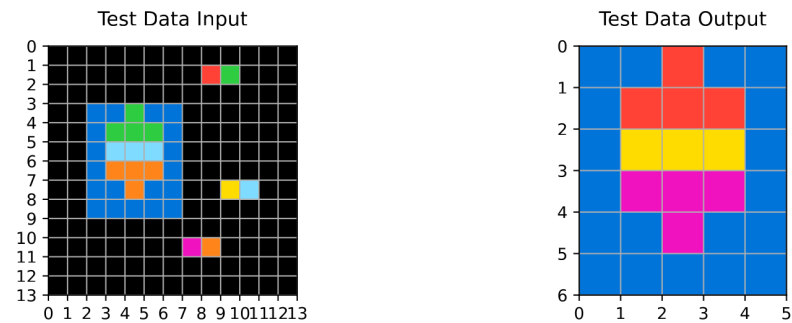


Figure 7—Visual for problem e9b4f6fc